

Contents

Week 2: Data Acquisition & Quality Control	2
AI/ML in Biomarker Discovery — miRNA in Alzheimer’s Disease	2
Learning Objectives	2
Conceptual Overview: Why QC and Normalization Matter	2
MODULE 2.1 — Understanding NCBI GEO	2
2.1.1 GEO Data Architecture	2
2.1.2 Selecting a Dataset: Evaluation Criteria	3
2.1.3 Our Working Datasets	3
MODULE 2.2 — Downloading GEO Data in R	4
2.2.1 The GEOquery Package	4
2.2.2 Downloading a GSE Record	4
2.2.3 Extracting the Expression Matrix	5
2.2.4 Extracting Sample Metadata	5
2.2.5 Downloading Supplementary Files (RNA-seq Counts for GSE46579)	6
MODULE 2.3 — Raw Data Formats	7
2.3.1 Microarray Data: Scanner Output and the Processed Matrix	7
2.3.2 Count Tables (RNA-seq)	8
2.3.3 SOFT Files	8
MODULE 2.4 — Quality Control for Microarray Data (GSE120584)	8
2.4.1 QC Metric 1 — Signal Distribution per Array	8
2.4.2 QC Metric 2 — Detected miRNAs per Sample	9
2.4.3 QC Metric 3 — Sample-to-Sample Correlation	9
2.4.4 Flagging and Removing Failed Samples	10
MODULE 2.5 — Filtering and Normalization of Microarray Data	10
2.5.1 Low-Expression Filtering	10
2.5.2 What Does Normalization Do?	11
2.5.3 Method 1: Reference-Gene Normalization — What GSE120584 Uses	11
2.5.4 Method 2: Quantile Normalization (and RMA)	12
2.5.5 Special Case: Hemolysis Detection for Blood miRNA	12
MODULE 2.6 — Quality Control for RNA-seq Count Data (GSE46579)	13
2.6.1 Loading the Count Table	13
2.6.2 QC Metric 1 — Library Size (Total Read Count per Sample)	14
2.6.3 QC Metric 2 — Detected miRNA Count	14
2.6.4 Low-Count Filtering	14
2.6.5 QC Metric 3 — Count Distribution	15
MODULE 2.7 — Normalization of RNA-seq Count Data (GSE46579)	15
2.7.1 Why Microarray Normalization Doesn’t Apply to Counts	15
2.7.2 Method 1: TMM (Trimmed Mean of M-values) — edgeR	15
2.7.3 Method 2: DESeq2 Median-of-Ratios	16
2.7.4 CPM, RPKM, TPM — What Not to Use (and Why)	16
MODULE 2.8 — Batch Effect Detection and Correction	17
2.8.1 What is a Batch Effect?	17
2.8.2 Detecting Batch Effects — PCA	17
2.8.3 Detecting Batch Effects — RLE Plot	18
2.8.4 Batch Correction — ComBat (sva package)	18
2.8.5 Batch Correction — limma::removeBatchEffect and SVA	19
MODULE 2.9 — The Clean Data Checkpoint	19
2.9.1 Pre-Analysis Data Audit Checklist	20
2.9.2 What a Clean Expression Matrix Should Look Like	20
WEEK 2 LAB SESSION	20
Lab 2A — Navigating GEO and Downloading Data (45 min)	20
Lab 2B — Quality Control Pipeline (60 min)	21
WEEK 2 ASSIGNMENTS	21
Reading Assignment	21

Reflection Questions	21
Practical Exercise	22
WEEK 2 GLOSSARY	22
KEY REFERENCES (Week 2)	23

Week 2: Data Acquisition & Quality Control

AI/ML in Biomarker Discovery — miRNA in Alzheimer’s Disease

Learning Objectives

By the end of Week 2, you will be able to: 1. Navigate NCBI GEO to identify, evaluate, and select appropriate miRNA datasets for AD biomarker analysis 2. Download and import GEO datasets programmatically into R using the GEOquery package 3. Understand the structure of microarray (processed log2 matrix, scanner/CEL files) and RNA-seq (count table) miRNA data 4. Extract and organize sample metadata (clinical phenotypes, covariates) from GEO records 5. Perform systematic quality control for both microarray (GSE120584) and RNA-seq (GSE46579) miRNA data 6. Apply appropriate normalization methods and understand the rationale behind each choice 7. Detect and correct batch effects using PCA, RLE plots, and ComBat/limma 8. Produce a clean, analysis-ready expression matrix with documented QC decisions

Conceptual Overview: Why QC and Normalization Matter

Imagine you are comparing blood glucose levels across 200 patients, but half the samples were measured with a calibrated analyzer and half with a cheap glucometer that reads 20% too high. Any “biological” difference you find between patient groups is contaminated by instrument artifact. The same problem exists in genomic data — at a much larger scale.

In a typical GEO dataset, samples may have been: - Processed in different laboratories or at different times (**batch effects**) - Extracted with different RNA isolation kits (**extraction efficiency variation**) - Hybridized to arrays on different days with different reagent lots (**technical variation**) - Collected from patients of different ages, sexes, or medication histories (**biological confounders**)

Quality control identifies samples that have failed technically.

Normalization removes systematic technical variation while preserving biological signal.

Batch correction removes variation attributable to processing groups rather than biology.

Done well, these steps produce an expression matrix where sample-to-sample differences reflect **true biological differences** between AD patients and controls — the signal we want our ML models to learn from.

MODULE 2.1 — Understanding NCBI GEO

2.1.1 GEO Data Architecture

NCBI GEO (Gene Expression Omnibus) organizes data in a hierarchical structure. Understanding this hierarchy is essential for navigating the database efficiently.

```

GEO Repository
|
├── GPL (Platform)
|   └── Describes the array or sequencer used

```

e.g., GPL16384 = Affymetrix Human Gene 2.1 ST Array

— GSM (Sample)

└ One biological sample; contains raw and/or processed data
e.g., GSM1234567 = serum from AD patient #001

— GSE (Series)

└ A complete study; links multiple GSMs and their GPL
e.g., GSE120584 = "Serum miRNA profiling in AD patients and controls"
Contains: study description, publication link, all GSM records, processed data files

— GDS (Dataset) – optional

└ Curated, analysis-ready subset created by NCBI staff
Not all GSEs have a GDS; use GSE directly when GDS unavailable

Key things to check in a GSE record before downloading:

Field	What to Look For
Summary	Study aims, disease, sample types, N per group
Overall Design	Experimental design, controls used, covariates measured
Contributor	Corresponding authors (helps assess study quality)
Platform (GPL)	Array type or sequencing platform; determines preprocessing workflow
Samples (GSM)	Number of samples; click individual GSMs to check metadata completeness
Supplementary Files	Raw data (CEL files), count matrices, processed expression tables
Linked Publications	PubMed IDs — always read the associated paper

2.1.2 Selecting a Dataset: Evaluation Criteria

Not all GEO datasets are equally suitable for our purposes. Use these criteria to evaluate datasets before committing to download:

Scientific criteria: - [] Disease: Alzheimer's disease (confirmed diagnosis, not just "dementia") - [] Biomarker type: miRNA (not mRNA, protein, or methylation) - [] Sample type: Blood-derived (serum, plasma, whole blood, PBMCs) - [] Has both AD patients AND healthy controls in the same study - [] Sample size: ≥ 20 per group (ideally ≥ 40 per group for stable ML training) - [] Metadata available: Age, sex, clinical stage (MCI vs clinical AD) are highly desirable

Technical criteria: - [] Platform is well-supported by Bioconductor (Affymetrix arrays, Illumina) - [] Normalization method is documented in associated paper - [] Spike-in controls or reference miRNAs documented (for RNA quantity normalization)

Red flags: - No control group (cannot perform differential expression) - Only processed/normalized data deposited with no raw data - Very small N (< 10 per group) — insufficient for ML

2.1.3 Our Working Datasets

For this course, we will work with the following AD miRNA datasets, selected for data quality, sample size, and biological relevance. Note that the two datasets were generated with **different technologies** — one of the most important lessons of this week is that each data type needs its own QC and normalization workflow.

Primary Dataset: GSE120584 - **Title:** Serum miRNA expression in dementia (Shigemizu et al., 2019, *Communications Biology*) - **Platform:** GPL21263 (Toray 3D-Gene Human miRNA Oligo Chip — a fluorescence **microarray**) - **Sample type:** Serum - **Groups:** 1,601 samples with five diagnoses — AD (1,021), MCI (32), normal cognition NC (288), vascular dementia VaD (91), dementia with Lewy bodies DLB (169). We use AD, MCI and NC (renamed “Control”). - **What GEO provides:** the submitters’ processed, normalized log2 matrix (`exprs()`), plus raw per-sample scanner files - **Why selected:** Very large sample size; includes an MCI class; linked to a peer-reviewed publication

Validation Dataset: GSE46579 - **Title:** A blood-based 12-miRNA signature of Alzheimer disease patients (Leidinger et al., 2013, *Genome Biology*) - **Platform:** GPL11154 (Illumina HiSeq 2000 — small **RNA-seq**) - **Sample type:** Whole blood - **Groups:** AD (n=48), controls (n=22) - **What GEO provides:** a supplementary Excel table of miRDeep2 read counts (raw and quantile-normalized); the series matrix itself contains no expression values - **Why selected:** Independent cohort for external validation (Week 5); a different platform *and* sample type tests how far a signature generalizes

	GSE120584 (primary)	GSE46579 (validation)
Technology	Fluorescence microarray	Small RNA sequencing
Values	Continuous log2 intensities	Integer read counts
Sample type	Serum (cell-free)	Whole blood (contains cells)
QC / normalization in this course	Detection-based QC; check submitters’ normalization	Library size, <code>filterByExpr</code> , DESeq2 VST / TMM
Differential expression (Week 4)	limma	DESeq2

Course convention: We will preprocess and clean each dataset independently, then use GSE120584 for model training and GSE46579 for external validation in Week 5.

Unbalanced cohort warning: In GSE120584, AD patients outnumber controls 3.5:1 and there are only 32 MCI samples. AD patients are also older (79 vs 72 years) and more often female (70% vs 48%). These imbalances shape every later week: covariates in Week 4, chance-level baselines in Week 3, and balanced performance metrics in Weeks 4–5.

MODULE 2.2 — Downloading GEO Data in R

2.2.1 The GEOquery Package

GEOquery is a Bioconductor package that provides programmatic access to all GEO records directly from R. It downloads the data, parses the SOFT file format, and returns structured R objects.

```
# Install if not already done (from Week 1 setup)
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")
BiocManager::install("GEOquery")
```

```
# Load the package
library(GEOquery)
```

2.2.2 Downloading a GSE Record

```
# Download the full GSE series
# destdir: where to save the files locally (avoids re-downloading)
# GSEMatrix: download the processed expression matrix (TRUE) or raw only (FALSE)
```

```

gse120584_list <- getGEO("GSE120584",
                        destdir   = "data/raw/",
                        GSEMatrix = TRUE,
                        AnnotGPL  = TRUE) # include probe annotations

# GEO often returns a list (one element per platform GPL)
length(gse120584_list)

# Extract the first (usually only) element
gse120584 <- gse120584_list[[1]]

# Inspect the object type
class(gse120584) # Should be "ExpressionSet"
dim(exprs(gse120584)) # 2562 probes × 1601 samples

```

What is an ExpressionSet?

An ExpressionSet is a standardized Bioconductor data container with three linked components:

```

ExpressionSet
├── exprs(gse120584)      – Expression matrix: rows = miRNAs, columns = samples
├── pData(gse120584)     – Phenotype data: rows = samples, columns = metadata fields
└── fData(gse120584)     – Feature data: rows = miRNAs, columns = probe annotations

```

This linked structure ensures that when you subset samples, all three components stay synchronized — a critical safety feature when working with metadata.

Microarray vs RNA-seq in GEO: For a microarray study, `exprs()` usually contains the depositors' processed matrix — here it is all we need. For an RNA-seq study such as GSE46579, `exprs()` is **empty** (0 rows): read counts are deposited as supplementary files instead (Section 2.2.5).

2.2.3 Extracting the Expression Matrix

```

# Extract the expression matrix
expr_raw <- exprs(gse120584)

# Basic inspection
dim(expr_raw)      # 2562 miRNAs × 1601 samples
range(expr_raw)    # about -1.9 to 16.1: already log2-transformed
sum(is.na(expr_raw)) # 0

# Rows are miRBase accessions (MIMAT...); swap in readable miRNA names
feature_data <- fData(gse120584)
head(feature_data[, c("ID", "miRNA_ID_LIST")])
rownames(expr_raw) <- feature_data$miRNA_ID_LIST
# A few probes detect several near-identical miRNAs,
# e.g. "hsa-miR-199a-3p, hsa-miR-199b-3p"

# How to read the value range:
# - Raw fluorescence: positive numbers, often in the hundreds or thousands
# - Processed: log2-transformed, normalized values (here about -2 to 16)

```

2.2.4 Extracting Sample Metadata

The phenotype data (`pData`) is often the most valuable and most poorly documented part of a GEO submission. Extracting it correctly is essential.

```
# Extract metadata table
metadata_raw <- pData(gse120584)

# Show all available metadata columns
colnames(metadata_raw)

# Common columns you will find in GEO:
# geo_accession      - GSM accession number
# title              - Sample name/identifier
# source_name_ch1    - Tissue/sample type (e.g., "serum")
# characteristics_ch1, characteristics_ch1.1, ... - Study-specific fields
# data_processing    - Normalization/processing steps applied - READ THIS

head(metadata_raw[, c("geo_accession", "title", "characteristics_ch1",
                     "characteristics_ch1.1", "characteristics_ch1.2")])
```

Parsing the characteristics columns: GEO encodes clinical metadata as free-text key:value pairs that need manual parsing. This is one of the most common sources of confusion for new users:

```
# View unique values to understand the encoding FIRST
unique(metadata_raw$characteristics_ch1)
# [1] "diagnosis: AD" "diagnosis: DLB" "diagnosis: MCI" "diagnosis: NC" "diagnosis: VaD"

metadata <- metadata_raw
metadata$diagnosis <- trimws(gsub("diagnosis: ", "", metadata$characteristics_ch1))

# Keep the three diagnoses this course studies
metadata <- metadata[metadata$diagnosis %in% c("NC", "MCI", "AD"), ]

# Rename to the labels used in every later week
group_labels <- c(NC = "Control", MCI = "Mild Cognitive Impairment", AD = "Alzheimer's Disease")
metadata$group <- factor(group_labels[metadata$diagnosis], levels = group_labels)

# Age ("age: 79") and sex ("Sex: female")
metadata$age <- as.numeric(gsub("age: ", "", metadata$characteristics_ch1.1))
metadata$sex <- trimws(gsub("Sex: |sex: ", "", metadata$characteristics_ch1.2))

# Summary of your cohort
table(metadata$group)           # Control 288 | MCI 32 | AD 1021
table(metadata$sex, metadata$group)
tapply(metadata$age, metadata$group, mean) # 71.7 | 75.5 | 79.2
```

Biological check: Before any analysis, verify that the cohort composition makes sense. Expected age distribution for AD: typically 65–90 years. Expected sex distribution: more females in AD cohorts (reflecting population demographics). Obvious anomalies (e.g., mean age 35 in an AD cohort) signal a metadata parsing error. Also compare the groups with each other — here the imbalance in size, age and sex is real and must be carried forward.

2.2.5 Downloading Supplementary Files (RNA-seq Counts for GSE46579)

For RNA-seq studies, GEO stores the read counts as supplementary files. GSE46579 deposits one Excel workbook with two sheets — raw counts and the submitters' quantile-normalized counts. We use the **raw** counts, because DESeq2 and edgeR need raw counts and normalize them with their own count-aware methods.

```
library(readxl)

gse46579 <- getGEO("GSE46579", destdir = "data/raw/", GSEMatrix = TRUE)[[1]]
dim(exprs(gse46579)) # 0 x 70 – no expression values in the series matrix

getGEOSuppFiles("GSE46579", makeDirectory = TRUE, baseDir = "data/raw/")
list.files("data/raw/GSE46579/")
# "GSE46579_AD_ngs_data_summarized.xls.gz"

# readxl cannot open .gz files: decompress a copy first
R.utils::gunzip("data/raw/GSE46579/GSE46579_AD_ngs_data_summarized.xls.gz",
               remove = FALSE)
excel_sheets("data/raw/GSE46579/GSE46579_AD_ngs_data_summarized.xls")
# "raw data" "normalized"
```

Supplementary archives (.tar): Many GEO series also provide a GSE..._RAW.tar archive with one file per sample (for GSE120584: 1,601 raw scanner files, GSM*.txt.gz). read.table() cannot read a .tar archive directly — extract it first with untar() (or tar -xvf in a terminal). We do not need these raw files in this course, because the processed matrix from exprs() is well documented.

MODULE 2.3 — Raw Data Formats

Understanding data formats prevents misinterpretation. This module explains what is actually inside the files you download.

2.3.1 Microarray Data: Scanner Output and the Processed Matrix

A fluorescence microarray measures mRNA abundance as the light emitted by labelled RNA bound to probes on a glass chip. For each sample the scanner produces a table of spots:

Raw 3D-Gene scanner file (one per sample, GSE120584_RAW.tar):

Cell	Block	Column	Row	G_Name	G_ID	635nm	Flag
1	1	1	1	hsa-miR-28-3p	MIMAT0004502	77.8	OK
1	1	2	1	hsa-miR-27a-5p	MIMAT0004501	71.0	OK
... (3,200 spots per array: miRNA probes, BLANK spots, negative and spike-in controls)							

The submitters processed these files before depositing the GEO matrix (read the data_processing field in pData()):

1. **Background subtraction:** background = mean + 2 SD of negative-control spots. Signals above background are “effective signals” and are log2-transformed.
2. **“Not detected” coding:** signals below background are **not** set to zero — they are replaced by a floor value (the minimum effective signal – 0.1). In every sample the lowest value is therefore the “not detected” floor, and many miRNAs sit exactly on it.
3. **Normalization:** each array is scaled to the mean of three internal-control miRNAs (miR-149-3p, miR-2861, miR-4463) that were stable in >500 serum samples.

Key consequences for analysis: - Values are continuous log2 intensities — no counts, no library sizes
 - A microarray never reports zero: “absent” means “at the floor” - Fold changes are **compressed**: background and a limited dynamic range make array differences smaller than the underlying biological differences (Week 4)

Affymetrix CEL files: Many other miRNA array datasets deposit raw Affymetrix **CEL** files, which store probe-level intensities. Those are processed with the oligo/affy packages and **RMA** (background correction, quantile normalization, summarization of several probes into one value per miRNA), with QC metrics such as NUSE. The principles

below — check distributions, detection and correlation before trusting the data — apply to every array platform.

2.3.2 Count Tables (RNA-seq)

For small RNA-seq data deposited in GEO, raw FASTQ files are usually hosted on SRA (Sequence Read Archive) — downloading and aligning them requires a high-performance computing environment beyond this course scope. Instead, GEO depositors typically also provide **processed count tables**. The GSE46579 table (from the miRDeep2 pipeline, miRBase v18) looks like this:

	AD	AD.1	AD.2	...	control	control.1	...
hsa-mir-30a:hsa-miR-30a-3p	115	156	193		...		
hsa-mir-550a-1:hsa-miR-550a-3p	241	931	415		...		
hsa-mir-29a:hsa-miR-29a-3p	29	91	42		...		
hsa-mir-155:hsa-miR-155-5p	21	19	16		...		
brain-mir-190:brain-mir-190	...	(novel miRNA predictions)					

- Rows = **precursor:mature** miRNA names; columns = samples named “AD”, “AD.1”, ..., “control”, ...
- Values = integer read counts (how many sequencing reads mapped to each miRNA)
- Column names match the GEO sample titles (“AD sample 0”, “AD sample 1”, ...), which we use to attach GSM accession numbers
- Two clean-up steps are needed (Week 2 script, Section 16C):
 - drop the 51 novel brain-mir-... predictions, keeping known human miRNAs
 - the same mature miRNA can come from several precursors (e.g. miR-16-5p from mir-16-1 and mir-16-2); miRDeep2 reports the same reads under each, so we keep the **maximum**, not the sum, per mature miRNA (86 duplicate rows)

Result: a matrix of **366 miRNAs × 70 samples**.

Key distinction from microarray: count data are **non-negative integers** with a characteristic **overdispersion** (variance > mean) that requires specific statistical methods (negative binomial models in DESeq2/edgeR) rather than methods assuming normally distributed data.

2.3.3 SOFT Files

GEO SOFT (Simple Omnibus Format in Text) files are the primary metadata format for GEO entries. They contain: - Platform (GPL) description: probe sequences and annotations - Sample (GSM) records: metadata and processed values for each sample - Series (GSE) record: study summary and design

GEOquery parses SOFT files automatically — you rarely need to handle them directly.

MODULE 2.4 — Quality Control for Microarray Data (GSE120584)

Quality control for microarray data aims to identify samples with technical failures: poor RNA quality, insufficient hybridization, damaged arrays or pipetting errors. Because GSE120584 comes already background-corrected and normalized, our QC works on the processed log2 matrix.

2.4.1 QC Metric 1 — Signal Distribution per Array

What it measures: The distribution of log2 expression values across all miRNAs for each array.

```
group_colours_vec <- GROUP_COLOURS[as.character(metadata$group)]
ord <- order(metadata$group) # plot samples grouped by diagnosis

boxplot(expr_raw[, ord],
```

```

main    = "Normalised log2 Expression per Array (GSE120584)",
ylab    = "log2 expression",
col     = group_colours_vec[ord],
border  = group_colours_vec[ord],
xaxt    = "n",                      # >1,000 sample names are unreadable
outline = FALSE)

```

```

median_signal <- apply(expr_raw, 2, median)
summary(median_signal)

```

Interpretation: - Boxes at very different positions → technical variation between arrays - One box clearly lower than all others → possible failed array - Medians are low here (≈ 2.3) because most of the 2,562 miRNAs are not detected in serum — the median is often the “not detected” floor. Module 2.5 re-checks the distributions after filtering.

2.4.2 QC Metric 2 — Detected miRNAs per Sample

What it measures: How many miRNAs rise above background in each sample. A sample in which very few miRNAs are detected may have had little RNA or a failed hybridization.

```

# Detected = above that sample's "not detected" floor (its minimum value)
sample_floor    <- apply(expr_raw, 2, min)
detected        <- sweep(expr_raw, 2, sample_floor + 0.01, ">") # TRUE/FALSE
detected_per_sample <- colSums(detected)
summary(detected_per_sample)    # range 379–2088, median 1239

# Flag samples far below the typical level (median - 3 MAD)
low_detect_cut  <- median(detected_per_sample) - 3 * mad(detected_per_sample)
low_detect_flag <- detected_per_sample < low_detect_cut

# Compare the groups
tapply(detected_per_sample, metadata$group, median)
# Control 1122 | MCI 1300 | AD 1264

```

A warning sign worth remembering: AD and MCI samples detect noticeably more miRNAs than Controls. A systematic group difference in a *technical* quality measure is more likely to reflect sample handling, storage time or array batch than biology — and it can later masquerade as “disease signal” (you will see its fingerprint in Weeks 3 and 4).

2.4.3 QC Metric 3 — Sample-to-Sample Correlation

What it measures: Pearson correlation between all pairs of samples based on their global expression profiles. Biologically similar samples should be more correlated with each other than with different groups — but all samples should show reasonably high correlation (typically > 0.90 for same-platform data).

```

library(pheatmap)
library(RColorBrewer)

cor_matrix <- cor(expr_norm, method = "pearson") # after filtering (Module 2.5)

# Typical agreement of each sample with all the others
median_cor <- apply(cor_matrix, 2, function(x) median(x[x < 1]))
low_cor_flag <- median_cor < 0.90
summary(median_cor)    # median 0.954; 13 samples below 0.90

```

```

annotation_col <- data.frame(Group = metadata$group, Sex = metadata$sex,
                             row.names = colnames(cor_matrix))

pheatmap(cor_matrix,
         annotation_col = annotation_col,
         color          = colorRampPalette(rev(brewer.pal(9, "RdBu")))(100),
         breaks         = seq(0.80, 1.0, length.out = 101),
         show_rownames  = FALSE, show_colnames = FALSE,
         filename       = "qc_reports/correlation_heatmap_GSE120584.png")

```

What to look for: - Clusters of samples from the same group (confirms biological signal exists) - Any sample with uniformly low correlation to all others (< 0.90) $\rightarrow$ likely failed sample - Strong clustering by sex, age, or processing batch rather than disease group $\rightarrow$ confounders need addressing

Choosing a threshold: A data-driven cut-off (median $- 3$ MAD) would flag 70 samples here, because correlations are tightly clustered around 0.95 — too aggressive. A fixed, interpretable threshold of 0.90 flags 13. Always look at the distribution before choosing, and report the rule you used.

2.4.4 Flagging and Removing Failed Samples

```

qc_log <- data.frame(
  sample      = metadata$geo_accession,
  group       = metadata$group,
  detected_miRNAs = detected_per_sample,
  median_cor  = median_cor,
  low_detection = low_detect_flag,
  low_correlation = low_cor_flag,
  pass_qc     = TRUE,
  exclude_reason = ""
)

qc_log$pass_qc[qc_log$low_detection] <- FALSE
qc_log$exclude_reason[qc_log$low_detection] <- "Detected miRNAs < median - 3 MAD"
qc_log$exclude_reason[qc_log$low_correlation & qc_log$pass_qc] <- "Median correlation < 0.90"
qc_log$pass_qc[qc_log$low_correlation] <- FALSE

table(qc_log$pass_qc, qc_log$group)
write.csv(qc_log, "qc_reports/sample_qc_decisions_GSE120584.csv", row.names = FALSE)

```

Result for GSE120584: no sample fails the detection rule; 13 fail the correlation rule (12 AD, 1 Control). **1,328 samples** remain: 287 Control, 32 MCI, 1,009 AD.

Documentation principle: Every sample removal decision must be documented with its reason. In a published paper or thesis, you will need to report: “N samples were excluded due to [reason]. Final analysis included N AD patients, N MCI patients, and N controls.”

MODULE 2.5 — Filtering and Normalization of Microarray Data

2.5.1 Low-Expression Filtering

miRNAs that sit below background in most samples contribute only noise and inflate the multiple-testing burden. We keep a miRNA if it is detected in at least 80% of the samples of **at least one**

group — checking group by group (the same logic as edgeR's `filterByExpr()` for RNA-seq) avoids discarding miRNAs that are switched on in only one diagnosis.

```
min_detect_rate <- 0.80
```

```
detect_rate_by_group <- sapply(levels(metadata$group), function(g)
  rowMeans(detected[, metadata$group == g, drop = FALSE]))
```

```
keep <- apply(detect_rate_by_group >= min_detect_rate, 1, any)
expr_filtered <- expr_raw[keep, ]
# 2562 → 925 miRNAs (1637 removed)
```

Rule: Never filter based on differential expression status (e.g., “keep only miRNAs with $p < 0.1$ ”). This introduces selection bias. Filter based on expression level and detection rate only.

2.5.2 What Does Normalization Do?

Normalization is a mathematical transformation that removes technical variation between samples while preserving biological differences. For miRNA microarray data, technical variation arises from: - Differences in total RNA input between samples - Differences in RNA labeling efficiency - Differences in hybridization conditions (temperature fluctuations, reagent lot) - Differences in scanner calibration

The central assumption of most normalization methods: the **majority of miRNAs are not differentially expressed** between groups. Therefore, differences in the bulk distribution of intensities are technical, not biological. This assumption is generally reasonable for blood miRNA in AD (most miRNAs do not change; only a subset are dysregulated).

Important caveat: If you are studying a biological condition where **global** expression changes are expected (e.g., comparing cells where you've knocked out a transcription factor that regulates most genes), standard normalization can erase true signal.

2.5.3 Method 1: Reference-Gene Normalization — What GSE120584 Uses

Reference-gene-based (RGB) normalization uses **stably expressed reference miRNAs** (analogous to housekeeping genes like GAPDH in RT-qPCR) as internal controls. Each sample's expression values are scaled relative to its reference miRNA levels. **This is exactly what the GSE120584 submitters did**, using three internal-control miRNAs (miR-149-3p, miR-2861, miR-4463).

Based on an article retrieved from PubMed, Wang et al. (2015) *Molecular BioSystems* (DOI: [10.1039/c4mb00711e](https://doi.org/10.1039/c4mb00711e)) systematically compared normalization methods for miRNA microarray data — including quantile, variance stabilization, robust spline, global scaling, and reference-gene approaches. Their key finding: **reference-gene normalization generally outperforms global methods**, particularly in biological conditions with large shifts in miRNA expression patterns, because it avoids “flattening” genuine large-scale differences.

Common reference miRNAs for blood-based studies: - **miR-93-5p** — frequently stable in serum - **miR-191-5p** — commonly used blood reference - **miR-16-5p** — platelet-derived; stable in plasma but unstable in serum if platelet contamination varies - **Spike-in controls (cel-miR-39, cel-miR-54)** — exogenous *C. elegans* miRNAs spiked in at a defined concentration during RNA extraction; best normalization control when available

The principle, shown with miR-93-5p as the reference:

```
ref_miR <- "hsa-miR-93-5p"
ref_vals <- expr_filtered[ref_miR, ]
```

```
# log2 scale: subtracting the reference = dividing by it
scaling_factors <- ref_vals - mean(ref_vals)
expr_rgb <- sweep(expr_filtered, 2, scaling_factors, "-")
```

```
# The reference miRNA is now constant across all samples
summary(expr_rgb[ref_miR, ])
```

Our job for GSE120584 is to check that the submitters' normalization worked:

```
median_filtered <- apply(expr_filtered, 2, median)
summary(median_filtered) # 3.97 - 6.22, median 4.92
round(tapply(median_filtered, metadata$group, median), 2) # 4.82 | 5.00 | 4.94
```

After filtering, per-array medians are similar across arrays and groups, so we use the submitters' normalization as provided.

2.5.4 Method 2: Quantile Normalization (and RMA)

Quantile normalization forces the distribution of intensities across all arrays to be **identical** — same minimum, same maximum, same median, same IQR. Each array's values are ranked, and each rank is replaced by the average value at that rank across all arrays. After quantile normalization, a box plot of all samples is completely flat.

Quantile normalization is also step 2 of **RMA** (Robust Multi-array Average), the standard pipeline for raw Affymetrix CEL files: (1) background correction, (2) quantile normalization, (3) summarization of each miRNA's probes by median polish.

```
library(limma)

# Optional in the Week 2 script (Section 8B): switched off by default
apply_quantile <- FALSE
if (apply_quantile) {
  expr_norm <- normalizeBetweenArrays(expr_filtered, method = "quantile")
} else {
  expr_norm <- expr_filtered # keep the submitters' normalization
}
```

When to use it: only if arrays remain clearly shifted after the depositor's normalization. Because it forces identical distributions, quantile normalization can erase genuine global differences between groups — the reason Wang et al. (2015) prefer reference-gene methods.

2.5.5 Special Case: Hemolysis Detection for Blood miRNA

A critical pre-analytical variable in blood miRNA studies is **hemolysis** — the lysis of red blood cells during or after blood collection, which releases miRNAs that are highly abundant in erythrocytes (particularly miR-451a and miR-23a-3p) and contaminates the serum/plasma miRNA profile.

Based on articles retrieved from PubMed, Murray et al. (2018) *Cancer Epidemiology, Biomarkers & Prevention* (DOI: [10.1158/1055-9965.EPI-17-0657](https://doi.org/10.1158/1055-9965.EPI-17-0657)) systematically characterized how pre-analytical variables including hemolysis and blood storage time affect circulating miRNA levels. Their key findings: - Levels of housekeeping miRNAs gradually increase over 14 days of storage at room temperature, in parallel with the hemolysis marker **hsa-miR-451a** - Normalizing to miR-451a can stabilize these storage-induced changes - Serum prepared with a low-speed centrifugation step is more suitable for miRNA quantification than plasma prepared for ctDNA extraction

Hemolysis detection:

```

# The ratio miR-451a / miR-23a-3p serves as a hemolysis index
mir451a_row <- grep("hsa-miR-451a$", rownames(expr_norm), value = TRUE)[1]
mir23a_row  <- grep("hsa-miR-23a-3p$", rownames(expr_norm), value = TRUE)[1]

if (!is.na(mir451a_row) && !is.na(mir23a_row)) {
  # log2 space: subtraction = log2 ratio
  metadata$hemolysis_index <- expr_norm[mir451a_row, ] - expr_norm[mir23a_row, ]
  metadata$hemolyzed       <- metadata$hemolysis_index > 7 # platform-dependent guide
  table(metadata$hemolyzed, metadata$group)
} else {
  cat("miR-451a or miR-23a-3p not found – hemolysis check skipped\n")
}

```

In our datasets: The GSE120584 submitters did not include miR-451a in the processed matrix, so the check cannot be run (the code reports this and moves on). GSE46579 is **whole blood**, which contains the red cells themselves — red-cell miRNAs (miR-486-5p, miR-92a-3p, miR-451a) make up ~95% of its reads by design — so the serum hemolysis index does not apply there (Module 2.6).

Why this matters clinically: If you build an ML model on data where AD patients happen to have slightly more hemolyzed samples than controls (due to sample handling differences), your model may be learning hemolysis signal, not disease biology. This would be a spurious biomarker that fails in prospective clinical validation.

MODULE 2.6 — Quality Control for RNA-seq Count Data (GSE46579)

For RNA-seq datasets from GEO, we typically work with pre-aligned count tables (since aligning FASTQ files requires HPC resources). QC of count data differs from microarray QC.

2.6.1 Loading the Count Table

```

library(readxl)

raw_counts_46 <- as.data.frame(
  read_excel(xls_file, sheet = "raw data", .name_repair = "minimal"))

row_ids <- raw_counts_46[[1]] # "precursor:mature" names
counts_46 <- as.matrix(raw_counts_46[, -1])

# Column names ("AD.12") → GEO titles ("AD sample 12") → GSM accessions
col_titles <- sub("^(AD|control)$", "\\1.0", colnames(counts_46))
col_titles <- sub("^(AD|control)\\.([0-9]+)$", "\\1 sample \\2", col_titles)
colnames(counts_46) <- metadata_46$geo_accession[match(col_titles, metadata_46$title)]

# One row per mature human miRNA (max over precursors – see Module 2.3.2)
mature <- sub(".*:", "", row_ids)
is_human <- grepl("^hsa-", mature)
count_matrix_46 <- apply(counts_46[is_human, ], 2,
  function(x) tapply(x, mature[is_human], max))

# Ensure sample order matches metadata
count_matrix_46 <- count_matrix_46[, metadata_46$geo_accession]
dim(count_matrix_46) # 366 miRNAs × 70 samples

```

2.6.2 QC Metric 1 — Library Size (Total Read Count per Sample)

```
library_sizes <- colSums(count_matrix_46)

barplot(library_sizes / 1e6,
        main = "Library Size per Sample (GSE46579)",
        ylab = "Total miRNA Reads (millions)",
        col = GROUP_COLOURS[as.character(metadata_46$group)],
        las = 2, cex.names = 0.55, border = NA)
abline(h = 0.5 * mean(library_sizes / 1e6), col = "red", lty = 2) # < 50% of mean

summary(library_sizes / 1e6)      # 1.3 – 52.6 million, median 15.3 million
```

What to look for: - Samples with very low library size (< 500,000 reads) → likely insufficient RNA or poor sequencing - Samples with library size < 50% of the cohort mean → flag for possible exclusion (5 samples here) - Library size variation > 5-fold across samples → strong normalization required (here > 40-fold)

Which miRNAs make up the reads?

```
read_share <- sweep(count_matrix_46, 2, library_sizes, "/") * 100
round(sort(rowMeans(read_share), decreasing = TRUE)[1:5], 1)
# miR-486-5p 90.5 | miR-92a-3p 4.0 | miR-451a 0.9 | miR-191-5p 0.8 | miR-182-5p 0.4
```

Whole blood is dominated by red-cell miRNAs. miR-486-5p alone takes ~90% of all reads in every sample. “Library size” therefore mostly measures one miRNA, and the reads left for all other miRNAs vary a lot between samples. This **composition effect** is why count-aware normalization (TMM, median-of-ratios) is essential here (Module 2.7).

2.6.3 QC Metric 2 — Detected miRNA Count

```
# How many miRNAs have at least 1 read in each sample?
detected_46 <- colSums(count_matrix_46 > 0)
summary(detected_46)      # 189 – 366, median ~350

barplot(detected_46,
        main = "Detected miRNAs per Sample (count > 0)",
        col = GROUP_COLOURS[as.character(metadata_46$group)],
        las = 2, cex.names = 0.55, border = NA)
```

2.6.4 Low-Count Filtering

Low-count miRNAs introduce noise without informative signal and inflate the multiple testing burden. They should be removed before normalization.

```
library(edgeR)

# Keep miRNAs with enough counts in at least as many samples as the smallest group
keep_46 <- filterByExpr(count_matrix_46,
                        group      = metadata_46$group,
                        min.count  = 10,
                        min.total.count = 15)

count_filtered_46 <- count_matrix_46[keep_46, ]
# 366 → 273 miRNAs (25% removed)
```

Only a quarter of the miRNAs are removed — far fewer than is typical for serum RNA-seq — because the submitters had already excluded miRNAs with fewer than 50 reads per group before depositing the table. Always read the data_processing notes to know what was done before you.

Rule: Never filter based on differential expression status (e.g., “keep only miRNAs with $p < 0.1$ ”). This introduces selection bias. Filter based on expression level and detection rate only.

2.6.5 QC Metric 3 — Count Distribution

```
# Raw counts are highly skewed; log-transform for visualization
# Add 0.5 pseudocount before log to handle zeros
log_counts <- log2(count_matrix_46 + 0.5)

boxplot(log_counts,
        main = "log2(count + 0.5) Distribution per Sample",
        col = GROUP_COLOURS[as.character(metadata_46$group)],
        las = 2, ylab = "log2(count + 0.5)", cex.axis = 0.55, outline = FALSE)
```

MODULE 2.7 — Normalization of RNA-seq Count Data (GSE46579)

2.7.1 Why Microarray Normalization Doesn't Apply to Counts

Count data differs from microarray intensity data in important ways: - Counts are **non-negative integers** (cannot be negative; many zeros) - Count variability scales with expression level (**mean-variance relationship**) - Counts have **overdispersion**: variance > mean (negative binomial distribution fits best) - Library size (total reads) and **composition** (a few miRNAs taking most reads) dominate technical variation

Methods designed for normally distributed data (like quantile normalization used for arrays) are **not appropriate** for raw count data. Instead, we use count-aware normalization methods.

2.7.2 Method 1: TMM (Trimmed Mean of M-values) — edgeR

TMM normalizes by computing, for each sample, a **scaling factor** that accounts for differences in RNA composition between samples. It trims away the most extreme miRNAs (30% of M-values and 5% of A-values) before computing the normalization factor, making it robust to the presence of a few highly expressed miRNAs that would otherwise dominate the calculation.

```
dge_46 <- DGEList(counts = count_filtered_46, group = metadata_46$group)
dge_46 <- calcNormFactors(dge_46, method = "TMM")

summary(dge_46$samples$norm.factors)      # ≈ 0.21 – 3.2

# TMM-normalized log2 CPM (Counts Per Million) values
cpm_tmm_46 <- cpm(dge_46, normalized.lib.sizes = TRUE, log = TRUE, prior.count = 0.5)
```

Reading the factors: In most datasets TMM factors lie between 0.9 and 1.1. In GSE46579 they spread from about 0.2 to 3, because miR-486-5p's share of the reads differs between samples: when one miRNA takes more of the reads, every other miRNA gets fewer. Correcting this composition effect is exactly what TMM is for — wide factors here are **not** by themselves a sign of failed samples.

2.7.3 Method 2: DESeq2 Median-of-Ratios

DESeq2 uses a **median-of-ratios** normalization that computes a size factor for each sample by: 1. Calculating a geometric mean expression level for each miRNA across all samples 2. Dividing each sample's counts by those geometric means 3. Taking the median of these ratios as the sample's **size factor**

This approach is robust to outlier miRNAs (a single very abundant miRNA such as miR-486-5p doesn't distort the size factor) and works well with count data.

```
library(DESeq2)
```

```
dds_46 <- DESeqDataSetFromMatrix(countData = count_filtered_46,  
                                colData   = metadata_46,  
                                design    = ~ group) # Week 4 adds ~ sex + age + group  
dds_46$group <- relevel(dds_46$group, ref = "Control")  
  
dds_46 <- estimateSizeFactors(dds_46)  
summary(sizeFactors(dds_46)) # ≈ 0.03 – 8.1
```

Size factors span a much wider range than usual because sequencing depth differed a lot between samples (1-53 million reads). Very extreme values (< 0.2 or > 5) deserve a look in the QC log.

2.7.4 CPM, RPKM, TPM — What Not to Use (and Why)

Students sometimes see these metrics in papers and want to use them. Here is a brief clarification:

Metric	Formula	Use Case	Appropriate for miRNA?
CPM (Counts Per Million)	$\text{count} / \text{lib_size} \times 1e6$	Cross-sample comparison of detection rates	Yes (library size correction only) — but misleading when one miRNA dominates
RPKM/FPKM	$\text{CPM} / \text{gene_length_kb}$	mRNA; accounts for gene length	No — miRNAs are all ~22 nt; length normalization is meaningless
TPM	$\text{RPKM} / \text{sum(RPKM)} \times 1e6$	mRNA; sum-normalized	No — same reason as RPKM
TMM	edgeR calc-	Differential expression	Yes — recommended
log-CPM	NormFactors → cpm()		
DESeq2 rlog/VST	Regularized log / variance-stabilizing transformation	Visualization, PCA, clustering, ML	Yes — recommended

For our course: Use **VST-transformed values** (via DESeq2) for visualization, PCA and ML features of GSE46579. Raw counts go into the DESeq2 differential expression model directly (Week 4).

```
# Variance-Stabilizing Transformation (VST) – preferred for visualization/ML  
# blind = TRUE: don't use design info (unbiased QC)  
# vst() is a fast shortcut that needs > 1,000 features; a miRNA dataset has a  
# few hundred, so call the full varianceStabilizingTransformation() instead  
vst_46 <- varianceStabilizingTransformation(dds_46, blind = TRUE)  
expr_vst_46 <- assay(vst_46)  
range(expr_vst_46) # about -0.2 to 25 (miR-486-5p at the top)
```

```
# Or: regularized log (rlog) – an alternative for small sample sizes
# rlog_46 <- rlog(dds_46, blind = TRUE)
```

After VST, GSE46579 is QC'd like an array: sample correlation (1 sample below 0.90), PCA (PC1 28%, PC2 12%; AD and Control separate on both), and a red-cell check (red-cell miRNAs are ~96% of reads in both groups, so blood-cell composition is not an obvious confounder). Five samples with library size < 50% of the mean are excluded, leaving **273 miRNAs × 65 samples** (21 Control, 44 AD).

MODULE 2.8 — Batch Effect Detection and Correction

2.8.1 What is a Batch Effect?

A batch effect is **systematic technical variation** introduced by processing samples in different groups (batches). Common batch sources in miRNA studies:

- **Date of RNA extraction:** RNA degradation enzymes in lab air; reagent lot differences
- **Date of library preparation or array hybridization:** Operator skill variation, reagent aging
- **Sequencing run:** Lane-to-lane variation on sequencing instruments
- **Processing site:** Multi-site studies with different laboratory protocols
- **Freeze-thaw cycles:** Samples thawed different numbers of times

Batch effects are insidious because they can **mimic biological signals** if batches are confounded with biological groups — for example, if all AD samples were extracted in January and all controls in June.

2.8.2 Detecting Batch Effects — PCA

Principal Component Analysis (PCA) is the primary tool for batch effect visualization. It reduces the high-dimensional expression matrix to a small number of “principal components” that capture the most variance in the data, then we plot samples in 2D colored by both group and batch to see which explains more of the variance.

```
library(ggplot2)
```

```
# Compute PCA on transposed expression matrix (samples as rows)
pca_result <- prcomp(t(expr_norm), scale. = TRUE)
```

```
# Variance explained by each PC
var_explained <- (pca_result$sdev^2) / sum(pca_result$sdev^2) * 100
# GSE120584: PC1 17.4%, PC2 9.3%
```

```
pca_df <- data.frame(
  PC1 = pca_result$x[, 1],
  PC2 = pca_result$x[, 2],
  Group = metadata$group,
  Batch = metadata$batch,      # only if a batch column exists in your metadata
  Sex = metadata$sex,
  Age = metadata$age
)
```

```
# PCA colored by GROUP (biology)
p1 <- ggplot(pca_df, aes(x = PC1, y = PC2, color = Group, shape = Group)) +
  geom_point(size = 2, alpha = 0.6) +
```

```

scale_color_manual(values = GROUP_COLOURS) +
labs(title = "PCA: Colored by Disease Group",
      x = paste0("PC1 (", round(var_explained[1], 1), "% variance)"),
      y = paste0("PC2 (", round(var_explained[2], 1), "% variance)")) +
theme_bw()

# PCA colored by BATCH (technical)
p2 <- ggplot(pca_df, aes(x = PC1, y = PC2, color = Batch, shape = Group)) +
  geom_point(size = 2, alpha = 0.6) +
  labs(title = "PCA: Colored by Processing Batch") +
  theme_bw()

library(gridExtra)
grid.arrange(p1, p2, ncol = 2)

```

Interpretation guide:

What you see	What it means
PC1 separates groups (AD vs control)	Strong biological signal — good!
PC1 separates batches, not groups	Batch effect dominates — must correct
PC1 separates groups AND batches	Confounded — difficult; correction with caution
Random scatter regardless of group	No biological signal detected (at this level)
Outlier samples far from cluster	Sample failed QC; confirm with correlation / detection metrics

GSE120584: Controls sit mostly on one side of PC1 and AD/MCI on the other, with heavy overlap. The GEO metadata contains **no batch, plate or processing-date column**, so batch cannot be checked directly — the script uses a single placeholder batch and skips ComBat. If structure unrelated to diagnosis appears, use SVA (below). Week 3 shows that PC1 is strongly tied to each array's overall signal level — a hint of hidden technical variation.

2.8.3 Detecting Batch Effects — RLE Plot

After normalization, RLE (Relative Log Expression) plots reveal whether sample- or batch-specific shifts remain. For each miRNA, subtract its median across samples; each sample's box of RLE values should be centred on zero with a similar width.

```

row_medians <- apply(expr_final, 1, median)
rle_matrix <- sweep(expr_final, 1, row_medians, "-")

boxplot(rle_matrix[, order(metadata_qc$group)],
        col = GROUP_COLOURS[as.character(metadata_qc$group)][order(metadata_qc$group)],
        main = "RLE Post-Normalization",
        ylab = "RLE", xaxt = "n", ylim = c(-2, 2), outline = FALSE)
abline(h = 0, col = "red", lty = 2)

```

If boxes within the same batch (or group) are systematically shifted (all above or below zero relative to others), a batch correction — or a closer look at the biology — is needed.

2.8.4 Batch Correction — ComBat (sva package)

ComBat (Johnson et al., 2007) uses an empirical Bayes approach to estimate batch-specific mean and variance parameters for each miRNA, then adjusts the data to remove these batch-specific

effects. It was developed for microarray data and works on any normalized log-scale matrix (array log2 values or RNA-seq VST values).

```
library(sva)

# ComBat requires:
# - expression matrix (miRNAs x samples) – already normalized, log scale
# - batch vector (factor identifying which batch each sample belongs to)
# - optional: biological covariates to PRESERVE (mod matrix)

mod <- model.matrix(~ group, data = metadata_qc) # keep group differences intact

expr_combat <- ComBat(dat      = expr_qc,
                      batch    = metadata_qc$batch,
                      mod      = mod,
                      par.prior = TRUE,
                      prior.plots = FALSE)

# Verify: PCA should no longer separate by batch
```

2.8.5 Batch Correction — limma::removeBatchEffect and SVA

A simpler, linear model-based alternative to ComBat. Appropriate when batch is a simple blocking factor (e.g., extraction date) and the data are approximately normally distributed (post-normalization microarray or VST-transformed RNA-seq).

```
library(limma)

expr_batch_corrected <- removeBatchEffect(
  x      = expr_qc,
  batch  = metadata_qc$batch,
  design = model.matrix(~ group, data = metadata_qc) # preserve group
)

# Use for visualization; for DE, include batch in the model instead (Week 4)
```

When batch labels are missing, **surrogate variable analysis (SVA)** estimates hidden technical factors from the data; the surrogate variables are then added as covariates in the Week 4 model (Week 2 script, Section 13C).

When to use ComBat vs removeBatchEffect:

Scenario	Recommendation
Multiple batches, large study	ComBat (more robust empirical Bayes approach)
Two batches, small N	limma::removeBatchEffect (simpler, fewer assumptions)
Batch perfectly confounded with group	Neither — cannot correct. Flag this as a study limitation.
Batch not documented in metadata (GSE120584)	Use surrogate variable analysis (SVA) to estimate hidden batch

MODULE 2.9 — The Clean Data Checkpoint

Before moving to Week 3, your data should meet all of the following criteria:

2.9.1 Pre-Analysis Data Audit Checklist

Sample integrity: - [] Failed samples identified by detection, correlation and library-size rules (exclusions documented) - [] Hemolysis assessed where applicable (serum/plasma) or its absence explained - [] Sample metadata is complete: group, age, sex, any known covariates

Expression matrix: - [] Undetected / low-count miRNAs filtered (detection rate for the array; filterByExpr for RNA-seq) - [] Data is normalized (reference-gene / optional quantile for the array; TMM or DESeq2 for RNA-seq) - [] Values on a log2 scale for visualization and ML (array log2; RNA-seq VST) - [] Batch effects assessed via PCA and RLE - [] Batch correction applied if needed and documented

Metadata alignment: - [] Column order of expression matrix matches row order of metadata table - [] Group labels are factored with correct reference level (Control as reference) - [] All covariates for downstream regression (age, sex) are correctly typed (numeric vs factor)

Documentation: - [] QC decisions saved to qc_reports/sample_qc_decisions_GSE120584.csv and ..._GSE46579.csv - [] Final sample counts per group recorded - [] Processing steps recorded in R script (reproducible)

```
# GSE120584 – microarray: normalized log2 matrix (no counts exist)
saveRDS(expr_final, "data/processed/GSE120584_expr_clean.rds")
saveRDS(metadata_qc, "data/processed/GSE120584_metadata_clean.rds")
```

```
# GSE46579 – RNA-seq: VST matrix for ML, filtered counts + DESeq2 object for DE
saveRDS(expr_vst_46_qc, "data/processed/GSE46579_expr_vst.rds")
saveRDS(count_filtered_46_qc, "data/processed/GSE46579_counts_filtered.rds")
saveRDS(dds_46_qc, "data/processed/GSE46579_dds.rds")
saveRDS(metadata_46_qc, "data/processed/GSE46579_metadata_clean.rds")
```

2.9.2 What a Clean Expression Matrix Should Look Like

	GSE120584 (serum array)	GSE46579 (whole-blood RNA-seq)
Rows	925 miRNAs (detected in $\geq$ 80% of a group)	273 miRNAs (filterByExpr)
Columns	1,328 samples (287 Control, 32 MCI, 1,009 AD)	65 samples (21 Control, 44 AD)
Values	Normalized log2 intensity ($\approx$ -1 to 16)	VST ($\approx$ 0 to 25)
Distribution	Similar per-array medians after filtering	Similar boxes after VST
PCA	Partial Control vs AD/MCI separation on PC1	AD vs Control separation on PC1 and PC2

WEEK 2 LAB SESSION

Lab 2A – Navigating GEO and Downloading Data (45 min)

Task: Find, evaluate, and download GSE120584 and GSE46579 using both the GEO web interface and GEOquery.

Step-by-step: 1. Go to <https://www.ncbi.nlm.nih.gov/geo/> 2. Search: GSE120584, then GSE46579 3. On each GSE page, answer these questions (write them down): - What sample types were used? -

What platform (GPL) was used — array or sequencer? - How many samples in each group? - What is deposited: a processed matrix, raw files (scanner files, CEL, FASTQ), or count tables? - What publication is linked? 4. Click on 3 individual GSM records and read the `data_processing` field — what was done to the data before deposit? 5. Download using `GEQuery` as shown in Module 2.2

Deliverable: A completed cohort description table for both datasets (platform, sample type, N per group, age range, sex distribution)

Lab 2B — Quality Control Pipeline (60 min)

Using `Week2_DataAcquisition_QC.R` (or the notebook `Week2_DataAcquisition_QC.ipynb`, which contains the same code):

1. **GSE120584 (Sections 3-14):** extract the expression matrix and metadata; plot signal distributions and detected miRNAs per sample; filter undetected miRNAs; check the submitters' normalization; compute sample correlations; run PCA; build the QC log
2. **GSE46579 (Sections 15-19):** load the count table; check library sizes and the miR-486-5p share; filter with `filterByExpr()`; normalize with `DESeq2` VST and TMM; compute correlations and PCA; build the QC log
3. Save the clean objects (Section 20)

Questions to answer: - How many samples (if any) were excluded from each dataset, and why? - Which QC metric differs systematically between groups in GSE120584, and why is that a concern? - What is the median library size in GSE46579, and which miRNA dominates it? - Were batch effects detected? If so, what correction was applied? If not, why not?

WEEK 2 ASSIGNMENTS

Reading Assignment

1. **Murray et al. (2018)** — *“Future-Proofing” Blood Processing for Measurement of Circulating miRNAs* (DOI: [10.1158/1055-9965.EPI-17-0657](https://doi.org/10.1158/1055-9965.EPI-17-0657))
Focus on: Table 1 (pre-analytical variables), hemolysis detection method, serum vs plasma comparison
2. **Wang et al. (2015)** — *Optimal consistency in microRNA expression analysis using reference-gene-based normalization* (DOI: [10.1039/c4mb00711e](https://doi.org/10.1039/c4mb00711e))
Focus on: Figure 2 (comparison of normalization methods), criteria used to evaluate methods, recommendation

Reflection Questions

1. Why is it dangerous to apply quantile normalization to miRNA data from a disease condition where global upregulation or downregulation is expected? How does reference-gene normalization address this?
2. You receive a GEO dataset where all AD samples were processed in batch 1 and all controls in batch 2. Can batch correction rescue this dataset for biomarker discovery? Why or why not?
3. A collaborator sends you a count matrix where some samples have 200,000 total reads and others have 8,000,000 total reads. What normalization approach would you use, and which samples (if any) might you exclude before normalization? How would your answer change if one miRNA made up 90% of every library, as miR-486-5p does in GSE46579?

Practical Exercise

Using the GSE46579 part of the Lab 2B pipeline, deliberately skip the normalization step and make a box plot comparison of AD vs control samples on $\log_2(\text{count} + 0.5)$. Then repeat with the VST values. Write 2-3 sentences describing what changes and why this matters for the ML analysis in Week 4.

WEEK 2 GLOSSARY

Term	Definition
CEL file	Raw Affymetrix array data file; contains probe-level fluorescence intensities for one sample (not used by our datasets)
Detection floor	The value a microarray assigns to below-background ("not detected") signals; in GSE120584 each sample's minimum value
Reference-gene normalization	Scaling each sample to stably expressed reference or internal-control miRNAs; used by the GSE120584 submitters (miR-149-3p, miR-2861, miR-4463)
RMA	Robust Multi-array Average; standard 3-step normalization for raw Affymetrix CEL files (background correction + quantile normalization + summarization)
Quantile normalization	Forces every sample to have an identical value distribution; step 2 of RMA; optional for GSE120584
RLE plot	Relative Log Expression plot; diagnostic showing each sample's deviation from the cohort median; boxes should center on zero
NUSE	Normalized Unscaled Standard Error; probe-level QC metric for Affymetrix CEL data; samples with median NUSE > 1.10 may have failed hybridization
DGEList	edgeR data container for RNA-seq count data; holds count matrix, sample information, and normalization factors
DESeqDataSet	DESeq2 data container; stores counts, sample metadata, and experimental design formula
TMM	Trimmed Mean of M-values; edgeR normalization method robust to outlier expression genes
VST	Variance-Stabilizing Transformation; DESeq2 method that stabilizes variance across the expression range; preferred for visualization and ML
rlog	Regularized log transformation; DESeq2 alternative to VST; better for very small sample sizes
CPM	Counts Per Million; count divided by library size $\times 1,000,000$; accounts for sequencing depth
Size factor	DESeq2 per-sample normalization coefficient; accounts for library size and RNA composition
Batch effect	Systematic technical variation between groups of samples processed at different times or locations
ComBat	Empirical Bayes batch correction method from the sva R package; adjusts batch-specific mean and variance per feature
SVA	Surrogate Variable Analysis; method for estimating hidden (unannotated) batch variables
Hemolysis	Lysis of red blood cells releasing cell-type-specific miRNAs (notably miR-451a) that contaminate serum/plasma miRNA profiles

Term	Definition
Library size	Total number of sequencing reads in one RNA-seq sample; primary source of technical variation
Composition effect	When a few very abundant miRNAs (e.g. miR-486-5p in whole blood) take a varying share of reads, all other miRNAs appear to change; corrected by TMM / median-of-ratios
miRDeep2	Pipeline that maps small RNA-seq reads to miRBase and reports counts per precursor:mature miRNA; used for GSE46579
filterByExpr	edgeR function for filtering low-count features in a group-aware manner
Spike-in control	Synthetic RNA of defined sequence and amount (e.g., <i>C. elegans</i> cel-miR-39) added to samples during extraction for normalization control
pData	phenoData; metadata slot in an ExpressionSet containing sample-level clinical and technical information
ExpressionSet	Bioconductor data container that links expression matrix, sample metadata, and feature annotations

KEY REFERENCES (Week 2)

All references retrieved from PubMed.

1. Murray MJ et al. (2018). “Future-Proofing” Blood Processing for Measurement of Circulating miRNAs in Samples from Biobanks and Prospective Clinical Trials. *Cancer Epidemiol Biomarkers Prev* 27(2):208–218. DOI: [10.1158/1055-9965.EPI-17-0657](https://doi.org/10.1158/1055-9965.EPI-17-0657)
2. Wang X, Gardiner EJ, Cairns MJ (2015). Optimal consistency in microRNA expression analysis using reference-gene-based normalization. *Mol Biosyst* 11(5):1235–1240. DOI: [10.1039/c4mb00711e](https://doi.org/10.1039/c4mb00711e)
3. Jiang X et al. (2025). Integrative bulk and single-cell transcriptomic profiling identifies core gene networks in glioma [demonstrates ComBat batch correction workflow on GEO datasets]. *BMC Cancer* 26(1):84. DOI: [10.1186/s12885-025-15454-5](https://doi.org/10.1186/s12885-025-15454-5)

Dataset References:

4. Shigemizu D et al. (2019). Risk prediction models for dementia constructed by supervised principal component analysis using miRNA expression data. *Commun Biol* 2:77. DOI: [10.1038/s42003-019-0324-7](https://doi.org/10.1038/s42003-019-0324-7) — GSE120584
5. Leidinger P et al. (2013). A blood based 12-miRNA signature of Alzheimer disease patients. *Genome Biol* 14(7):R78. DOI: [10.1186/gb-2013-14-7-r78](https://doi.org/10.1186/gb-2013-14-7-r78) — GSE46579

Software and Methods References (key methods papers cited for tools used):

6. Irizarry RA et al. (2003). Exploration, normalization, and summaries of high density oligonucleotide array probe level data. *Biostatistics* 4(2):249–264. DOI: [10.1093/biostatistics/4.2.249](https://doi.org/10.1093/biostatistics/4.2.249) — RMA normalization
7. Robinson MD, McCarthy DJ, Smyth GK (2010). edgeR: a Bioconductor package for differential expression analysis of digital gene expression data. *Bioinformatics* 26(1):139–140. DOI: [10.1093/bioinformatics/btp616](https://doi.org/10.1093/bioinformatics/btp616) — edgeR / TMM normalization
8. Love MI, Huber W, Anders S (2014). Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biol* 15:550. DOI: [10.1186/s13059-014-0550-8](https://doi.org/10.1186/s13059-014-0550-8) — DESeq2 / size factor normalization

9. Johnson WE, Li C, Rabinovic A (2007). Adjusting batch effects in microarray expression data using empirical Bayes methods. *Biostatistics* 8(1):118-127. DOI: [10.1093/biostatistics/kxj037](https://doi.org/10.1093/biostatistics/kxj037) — *ComBat batch correction*
10. Ritchie ME et al. (2015). limma powers differential expression analyses for RNA-sequencing and microarray studies. *Nucleic Acids Res* 43(7):e47. DOI: [10.1093/nar/gkv007](https://doi.org/10.1093/nar/gkv007) — *limma / removeBatchEffect*

Next Week: Exploratory Data Analysis — We will apply descriptive statistics, PCA, clustering, heatmaps and confounder analysis to understand the structure of the clean GSE120584 data before any supervised machine learning.